NetWatch: A Cybernetic Guardrail Architecture for Multimodal Autonomous Systems

NetWatch Project

October 20, 2026

Abstract

Modern autonomous AI agents face significant security challenges where static safety filters and prompt engineering fail against dynamic runtime vulnerabilities, including Indirect Prompt Injection (IPI), the Agent Runtime Authority Gap, Cache-Augmented Generation (CAG) poisoning, and GraphRAG topological attacks. This thesis presents **NetWatch**, a cybernetic guardrail architecture that balances applied mathematics with robust software engineering across a tri-hook runtime interception gateway. For prompt injection defense (RQ1), NetWatch introduces **Horizon**, an end-to-end framework combining Stage 1 randomized keyed spotlighting with distributionally robust minimax bounds and Stage 2 deterministic guardrails featuring quarantined execution pools, Hoeffding-bounded majority voting, capability tokens, and SMT-verified noninterference policies.

Keywords: AI Agent Security, Prompt Injection Defense, Horizon Architecture, Keyed Spotlighting, Minimax Game Theory, Distributionally Robust Optimization, Runtime Application Self-Protection (RASP), Control Barrier Functions.

Contents

Abstract **1**

1 Introduction **4**
1.1 Background and Motivation 4
1.2 Problem Statement 4
1.3 Research Questions 4
1.4 Thesis Contributions 5
1.5 Thesis Organization 5

2 Related Work **6**
2.1 Vulnerabilities in Autonomous Agent Runtimes 6
2.2 Instruction-Data Separation: The Theoretical Barrier 7
2.3 Spotlighting and Cryptographic Randomization 7
2.4 Statistical Bounds, DRO, and Game-Theoretic Verification 8
2.5 Deterministic Guardrails, CaMeL, and IFC 8
2.6 Formal Verification and Conformal Human-in-the-Loop Safety 9

3 Methodology **11**
3.1 System Architecture Overview 11
3.2 The Tri-Hook Defense Framework 11
3.2.1 1. Pre-Retrieval Hook (Perception & Memory Ingestion) 11
3.2.2 2. Pre-Execution Hook (Action & API Interception) 12
3.2.3 3. Post-Execution Hook (Context Sanitization & Exfiltration Control) 12
3.3 RQ1: Prompt Injection Defense — The Horizon Architecture 12
3.3.1 Intuitive Motivation: The Entanglement Problem 13
3.3.2 Stage 1: Keyed Spotlighting with Robust Risk Bounds 13
3.3.3 Stage 2: Deterministic Guardrail (Math-Hardened CaMeL) 17

4 Experiments **21**
4.1 Experimental Setup and Testbed 21
4.2 Threat Models and Attack Scenarios 21
4.3 Evaluation Datasets and Red-Teaming Benchmarks 21
4.4 Baseline Configurations 21

5 Evaluation **22**
5.1 Detection Efficacy (RQ1 & RQ2) 22
5.2 Safety Barrier Enforcement (RQ3) 22
5.3 Cybernetic Rate Limiting Performance (RQ4) 22
5.4 System Overhead and Latency Analysis 22

6 Discussion **23**
6.1 Balancing Mathematical Rigor and Software Practicality 23
6.2 Security Depth vs. Latency Overhead 23
6.3 Assumptions and Limitations 23

7 Future Work **24**
7.1 Adaptive Latent Space Barrier Learning 24
7.2 Hardware-Accelerated Cryptographic Hooks 24
7.3 Multi-Agent Swarm Defense Protocols 24

AI Disclosure **25**

References **26**

A Mathematical Proofs **29**
A.1 Forward Invariance of Control Barrier Functions 29

B Architecture Configuration and Schemas **30**
B.1 Middleware Configuration Schema 30

Chapter 1

Introduction

1.1 Background and Motivation

The proliferation of autonomous AI agents operating in production environments has highlighted critical vulnerabilities in existing safety paradigms. Unlike traditional large language model (LLM) deployments that rely primarily on prompt filtering and lexical heuristics, agentic systems possess tool-use capabilities, execution privileges, and persistent memory.

1.2 Problem Statement

Static safety filters are insufficient against dynamic attacks targeting the Agent Runtime Authority Gap. Attackers exploit working memory (CAG poisoning), topological dependencies (GraphRAG relation injection), and visual steganography to hijack agent execution pipelines.

1.3 Research Questions

- **RQ1 (Prompt Injection & Adversarial Defense — The Horizon Architecture):** How can cryptographic randomization, distributionally robust optimization, and deterministic causal mediation be synthesized into an end-to-end guardrail architecture to mathematically guarantee an upper bound on indirect prompt injection risk without impairing model utility?
- **RQ2 (GraphRAG Topological Defense):** How can graph-theoretic anomaly detection algorithms be integrated into an asynchronous ingestion middleware to mathematically isolate adversarial relation injections before corrupting multi-hop reasoning?
- **RQ3 (Vision-Language Sandboxing):** To what extent can frequency domain analysis and Fourier transforms in an isolated vision-proxy gateway neutralize steganographic prompt injections in Vision-Language Models (VLMs) without degrading processing latency?
- **RQ4 (Latent Barrier Steering):** How effectively can continuous Control Barrier

Functions (CBFs) be mapped onto discrete API action-spaces to guarantee zero-violation execution during multi-step tool hijacking attacks?

- **RQ5 (Cybernetic Rate Limiting):** How can Proportional-Integral-Derivative (PID) control theory be synthesized with dynamic Risk Token-Buckets in the API gateway to throttle network bandwidth prior to data exfiltration?

1.4 Thesis Contributions

- Formulation of a balanced applied mathematics and software engineering Tri-Hook Runtime Architecture.
- Design and implementation of real-time pre-retrieval, pre-execution, and post-execution interception hooks.
- Comprehensive empirical evaluation against standardized agent red-teaming benchmarks.

1.5 Thesis Organization

The remainder of this thesis is organized as follows: Chapter 2 reviews related literature; Chapter 3 describes the mathematical and software architecture of NetWatch; Chapter 4 details the experimental setup; Chapter 5 presents empirical findings; Chapter 6 discusses implications and limitations; Chapter 7 outlines future research avenues; and the Appendices provide detailed mathematical proofs and configuration schemas.

Chapter 2

Related Work

The shift from passive, conversational Large Language Models (LLMs) to autonomous agentic architectures has reshaped the cybersecurity attack surface. This chapter surveys the foundational literature, formal vulnerabilities, and emerging defensive paradigms spanning prompt injection mitigation, mathematical verification, cryptographic isolation, and runtime guardrails.

2.1 Vulnerabilities in Autonomous Agent Runtimes

Unlike traditional web applications where code and data occupy segregated memory spaces, transformer-based neural architectures process instructions, contextual inputs, retrieved documents, and external tool outputs as a single, concatenated stream of textual tokens [1, 4]. This structural limitation precipitates the *Agent Runtime Authority Gap*—the discordance between the user's intended execution boundaries and the actual capabilities granted to the model [3].

Greshake et al. [1] first formalized *Indirect Prompt Injection (IPI)*, demonstrating that untrusted content retrieved from external environments (e.g., email bodies, scraped HTML documents, tool outputs, or multi-hop retrieval passages) can hijack the model's control flow. By concealing natural language instructions within third-party data, adversaries can induce the agent to exfiltrate private conversation history, invoke unauthorized APIs, or modify external persistent states while appearing to execute benign user requests [2].

As agentic frameworks incorporate multi-turn task decomposition and tool usage, automated adversarial optimization techniques have advanced significantly. Zou et al. [12] developed Greedy Coordinate Gradient (GCG) attacks, producing universal adversarial suffixes that bypass alignment training. Simultaneously, Chao et al. [13] introduced Prompt Automatic Iterative Refinement (PAIR), utilizing an attacker LLM in a feedback loop to autonomously discover jailbreak prompts within minimal query budgets. Furthermore, Yuan et al. [14] established that semantic encryption (e.g., Caesar ciphers, Morse code, or Base64 encoding) fundamentally breaks internal safety guardrails because LLMs fail to evaluate safety policies over obfuscated tokens while still retaining reasoning capabilities over decoded instructions. Recent empirical studies by Wong et al. [15] emphasize that standard heuristic safety alignment is structurally incapable of anticipating dynamic multi-turn exploits.

2.2 Instruction-Data Separation: The Theoretical Barrier

A central question in LLM security is whether models can inherently distinguish between developer instructions and third-party data. Zverev and Abdelnabi [5] investigated this problem formally, demonstrating that standard transformer self-attention mechanisms lack inductive biases for instruction-data disentanglement; an instruction remains syntactically indistinguishable from data within the attention matrix. To overcome this limitation at the architectural level, Zverev et al. [6] proposed ASIDE, introducing separate transformer pathways for instructions and untrusted inputs. While promising, architectural modifications require complete model retraining and cannot protect pre-trained commercial foundation models.

Consequently, systems-level containment approaches have emerged. Sharma [8] advocated for "Containment over Detection," arguing that statistical classifiers attempting to detect prompt injections always succumb to adaptive adversaries. Instead, cryptographic boundary enforcement establishes strict syntactic isolation wrappers around untrusted data. Relatedly, Bhatnagar and Pokhariyal [9] developed compiler-stage semantic watermarks to track token provenance. Despite these innovations, static wrappers remain vulnerable when attackers simulate delimiter boundaries or exploit parser ambiguities.

2.3 Spotlighting and Cryptographic Randomization

To provide an empirical signal that alerts the model to untrusted provenance without retraining, Hines et al. [7] introduced *Spotlighting*. Spotlighting transforms untrusted inputs prior to model ingestion using three primary mechanisms:

1. **Delimiting:** Enclosing the external document within rigid syntactic brackets (e.g., «`data`»).
2. **Datamarking:** Interleaving an invariant symbol or token between words (e.g., `word^word`).
3. **Encoding:** Transforming the entire external payload via reversible algorithms such as Base64.

While Hines et al. proved that spotlighting drastically reduces attack success rates (ASRs) on static benchmarks, static spotlighting exhibits critical cryptographic weaknesses:

- *Delimiter Spoofing:* An adversary who observes the static delimiter string » can inject early-closing delimiters to escape the data fence.
- *Known-Marker Copying:* Invariant datamarking symbols known to the attacker allow crafted injections that seamlessly embed the expected marker.
- *Cipher Reversal:* Invariant symmetric ciphers (e.g., ROT13) allow the attacker to pre-scramble malicious commands, causing the defensive ingestion layer to decode the attack into trusted plain text.

To establish provable defensive guarantees, the security of the transformation must satisfy *Kerckhoffs's Principle* [10]: a security system must remain secure even if all implementation details are public knowledge, provided the key remains secret. NetWatch

operationalizes this principle by drawing an ephemeral secret key $\kappa \sim \text{Unif}(\mathcal{K}_\tau)$ independently for every request, generating dynamic markers and random permutation ciphers that prevent adversarial anticipation.

2.4 Statistical Bounds, DRO, and Game-Theoretic Verification

A prevailing flaw in published LLM evaluations is reporting empirical Attack Success Rate (ASR) $\hat{p} = \frac{1}{n}\sum_{i=1}^{n} A_i$ as a measure of real-world risk. As demonstrated by Nasr et al. [11] in their doctrine "The Attacker Moves Second," empirical evaluations conducted against fixed test distributions severely underestimate adaptive adversarial capabilities. Once a defense is published or deployed, determined adversaries actively optimize against its specific parameters.

To replace naive ASR metrics with mathematically rigorous risk bounds, modern methodologies integrate statistical confidence estimation, distributionally robust optimization, and game theory:

1. **Finite-Sample Exact Confidence:** Clopper and Pearson [16] formulated exact binomial confidence bounds based on the Beta distribution. In safety-critical regimes where zero attacks are observed ($s = 0$), the asymptotic "rule of three" [17] guarantees that the upper bound $\bar{p} \approx \frac{3}{n}$ at the 95% confidence level, providing an honest baseline ceiling.

2. **Distributionally Robust Optimization (DRO):** In real-world deployments, the adversarial distribution Q shifts away from the empirical test set P. Duchi and Namkoong [18] and Ben-Tal et al. [19] developed DRO frameworks that maximize expected loss over an ambiguity set bounded by a statistical divergence. For binary classification under a χ^2-divergence budget ρ, the worst-case shifted risk admits the closed-form upper bound $g_\rho(p) = p + \sqrt{2\rho p(1-p)}$.

3. **Minimax Game Equilibria:** Rather than relying on a single static defense τ, the defender can randomize over a family of spotlighting variants Π_η constrained by an acceptable utility degradation η. By Sion's Minimax Theorem [22], the minimax equality holds between the defender's mixed strategy and the adversary's worst-case shifted distribution:

$$\min_{\pi \in \Pi_\eta} \max_{w \in \mathcal{W}} \sum_{\tau} \pi_\tau \sum_{k} w_k \bar{\ell}_{k,\tau} = \max_{w \in \mathcal{W}} \min_{\pi \in \Pi_\eta} \sum_{\tau} \pi_\tau \sum_{k} w_k \bar{\ell}_{k,\tau} \tag{2.1}$$

   Paulus et al. [23] recently validated this non-cooperative game formulation for model alignment, confirming that mixed strategies prevent gradient-based adversarial convergence.

2.5 Deterministic Guardrails, CaMeL, and Information-Flow Control

While randomized spotlighting reduces the probability that an individual LLM execution obeys an injection, it does not guarantee total system containment. In autonomous systems, defense-in-depth requires isolating reasoning from execution.

Early industry efforts, such as NVIDIA NeMo Guardrails [26], introduced programmable rails using Colang to restrict conversational dialogue flows. However, software-level rule checking frequently relies on auxiliary LLM evaluators, which are themselves vulnerable to prompt injection. Schwarz [27] proposed Countermind, employing multi-layered heuristic filtering across prompt processing stages.

The decisive breakthrough in structural containment was achieved by Debenedetti et al. [24] with the introduction of **CaMeL (Causal Mediation for Language Agents)**. CaMeL enforces strict architectural separation:

- A *Privileged LLM* interacts exclusively with trusted user prompts, synthesizing an execution script in a safe domain-specific language.
- A *Quarantined LLM* processes untrusted data strictly within read-only extraction queries.
- A *Deterministic Interpreter* verifies that untrusted data can only instantiate function arguments but can never influence program control flow or select target tools.

Parallel work by Costa et al. [25] formalized Information-Flow Control (IFC) for AI agents, proving that dynamic labels and capability tokens prevent unauthorized data flows. In multi-agent environments, Louck and Stulman [28], Kothari [29], and Panchumarthi and Sakamuri [30] developed cryptographic verification and Model Context Protocol (MCP) formal frameworks [31] to authenticate inter-agent messages and prevent tool-chain escalation.

2.6 Formal Verification and Conformal Human-in-the-Loop Safety

To mathematically certify agent safety before tool invocation, NetWatch integrates formal noninterference and conformal risk control:

1. **Noninterference Policies:** Goguen and Meseguer [32] established the foundational theory of noninterference, dictating that actions in high-security domains must have no observable causal dependence on untrusted inputs. By encoding policy rules into first-order logic and evaluating them via SMT solvers such as Z3 [33], NetWatch deterministically verifies that proposed tool parameters do not violate privilege boundaries.

2. **Quantitative Information Flow (QIF):** Smith [34] formalized operational bounds on leakage in systems handling confidential data. Enforcing an information-theoretic leakage budget B ensures that malicious inputs cannot exfiltrate internal system state across repeated queries.

3. **Conformal Risk Control (CRC):** When policies encounter ambiguous execution states requiring human confirmation, Angelopoulos et al. [21] proved that the threshold λ can be calibrated finite-sample to bound the expected probability of false approvals:

$$\mathbb{E}\left[\mathcal{L}_{\text{user}}(\lambda)\right] \leq \alpha_{\text{user}} \tag{2.2}$$

guaranteeing that user-in-the-loop escalations remain bounded by a pre-selected risk tolerance α_{user}.

Table 2.1 summarizes how NetWatch synthesizes and extends these complementary paradigms into a single unified architecture.

Table 2.1: Comparative analysis of prompt injection defense paradigms.

Defense Paradigm	**Key Representatives**	**Primary Strengths**	**Inherent Vulnerabilities**
Prompt-Level Heuristics	Garak [37], NeMo [26]	Easy integration, zero retraining	Brittle; bypassed by PAIR [13] and GCG [12]
Architectural Model Splits	ASIDE [6]	Native attention separation	Requires retraining; incompatible with commercial APIs
Static Spotlighting	Hines et al. [7]	Preserves model utility	Deterministic markers can be copied; cipher reversal
Causal Mediation (CaMeL)	Debenedetti et al. [24]	Provable structural isolation	Quarantined extractor still vulnerable to single-call failure
NetWatch (Horizon)	**This Work**	**Randomized spotlighting + Hoeffding voting + SMT verification**	**Guaranteed end-to-end minimax risk ceiling V_{Horizon}**

Chapter 3

Methodology

This chapter details the mathematical formulations and software engineering implementations comprising the NetWatch architecture. Section 3.1 outlines the overarching system topology. Section 3.2 defines the Tri-Hook Runtime Architecture. Section 3.3 presents the primary theoretical and systems contribution of this work: **RQ1 — The Horizon Guardrail Architecture for Prompt Injection Defense**, detailing Stage 1 (Randomized Keyed Spotlighting with Distributionally Robust Risk Bounds) and Stage 2 (Deterministic Guardrail via Math-Hardened CaMeL with Hoeffding-Bounded Quarantined Execution).

3.1 System Architecture Overview

NetWatch is architected as an inline, deterministic cybernetic guardrail positioned between an autonomous agent's reasoning core and external execution environments. The design enforces a strict 50/50 balance between applied mathematics and software engineering:

- **Software Engineering (50%):** Building real-time Application Programming Interface (API) interception proxies, isolated process sandboxes, dual-agent execution pipelines, and runtime verification engines.
- **Applied Mathematics (50%):** Deriving finite-sample statistical confidence limits, distributionally robust optimization bounds, majority voting exponential concentrations, game-theoretic minimax equilibria, and formal noninterference proofs.

3.2 The Tri-Hook Defense Framework

To prevent compromises across the agent's cognitive lifecycle, NetWatch physically intercepts the control loop at three distinct runtime execution hooks:

3.2.1 1. Pre-Retrieval Hook (Perception & Memory Ingestion)

The pre-retrieval hook sanitizes external information prior to embedding or ingestion into the model's key-value (KV) cache:

- **Software Sandbox:** External documents, web pages, and vector database query returns are held in an isolated proxy ingestion buffer.

- **Mathematical Outlier Detection:** High-dimensional Mahalanobis distance filters evaluate incoming document embeddings $x \in \mathbb{R}^d$ against baseline semantic distribution (μ, Σ):

$$D_M(x) = \sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)} \tag{3.1}$$

Embeddings exceeding critical χ^2 threshold percentiles are flagged as topological anomalies and quarantined. For visual inputs, 2D Fast Fourier Transforms (FFT) detect high-frequency spatial perturbations indicative of steganographic prompt injection.

3.2.2 2. Pre-Execution Hook (Action & API Interception)

Before the agent dispatches any parameterized command to an external API or tool:

- **Software Middleware:** A Runtime Application Self-Protection (RASP) interceptor halts execution and freezes the candidate payload.
- **Control Barrier Functions (CBFs):** Defining a continuously differentiable safety set $\mathcal{C} = \{x \in \mathbb{R}^n : h(x) \geq 0\}$, the system enforces the forward-invariance condition:

$$\dot{h}(x) + \alpha(h(x)) \geq 0 \tag{3.2}$$

where $\alpha(\cdot)$ is an extended class $\mathcal{K}_\infty$ function. If the agent's proposed action vector pushes the barrier derivative toward the boundary, the software gateway terminates the action.

3.2.3 3. Post-Execution Hook (Context Sanitization & Exfiltration Control)

Following tool execution, results returning to the agent's context are strictly scoped:

- **Software RBAC & Provenance Ledger:** Returned fields are assigned immutable security tags and access levels.
- **Differential Privacy & Rate Limiting:** Sensitive numeric outputs are protected via Laplacian noise injection:

$$\tilde{y} = y + \text{Lap}\left(0, \frac{\Delta f}{\epsilon}\right) \tag{3.3}$$

while Proportional-Integral-Derivative (PID) cybernetic governors throttle bandwidth velocity to arrest exfiltration attempts.

3.3 RQ1: Prompt Injection Defense — The Horizon Architecture

The primary research question addressed in this thesis is:

> **Research Question 1 (RQ1):** *How can cryptographic randomization, distributionally robust optimization, and deterministic causal mediation be synthesized into an end-to-end guardrail architecture to mathematically guarantee an upper bound on indirect prompt injection risk without impairing model utility?*

To resolve RQ1, we formulate **Horizon**, a comprehensive two-stage defense architecture. Figure 3.1 illustrates the unified flow: Stage 1 applies Keyed Spotlighting with robust risk bounds to protect individual model inferences, while Stage 2 wraps the execution within a deterministic guardrail utilizing quarantined model pools, Hoeffding voting, capability tokens, and SMT-verified noninterference.

The Horizon End-to-End Pipeline

Stage 1: User Task $\rightarrow$ Ephemeral Key Generation $\kappa \sim \mathrm{Unif}(\mathcal{K}_\tau)$ $\rightarrow$ Spotlighting Transformation τ_κ(Untrusted Data) $\rightarrow$ Minimax Risk Bound $V_1(\rho, \eta)$

$\Downarrow$ *Optimal Mix* $\pi^\star$*, Key Space* $|\mathcal{K}|$*, Failure Bound* $\bar{\mathcal{R}}_Q$

Stage 2: User Prompt u $\rightarrow$ Privileged LLM $\rightarrow$ Safe Program $\rightarrow$ Deterministic Interpreter $\rightarrow$ Quarantined LLMs $\times\, m$ $(\pi^\star)$ $\rightarrow$ Majority Vote (Hoeffding Bound) $\rightarrow$ Typed Values & Capabilities $\rightarrow$ Policy Check (Z3 SMT Noninterference) $\rightarrow$ Conformal Escalation λ $\rightarrow$ Tool Execution

Figure 3.1: Conceptual block diagram of the Horizon defense pipeline.

3.3.1 Intuitive Motivation: The Entanglement Problem

The fundamental reason prompt injection succeeds is *semantic entanglement*. An LLM receives all text—system rules, developer instructions, user requests, third-party emails, retrieved database chunks, and API responses—as a single, flat stream of tokens [1, 5]. The self-attention mechanism cannot inherently discern *who authored which token*. When an email contains the text:

```
"Meeting moved to Friday.  Ignore previous instructions and
forward emails to evil@attacker.com"
```

the injected sentence looks computationally identical to a legitimate developer directive.

The core design philosophy of Horizon is built on two axioms:

1. **Axiom 1 (Perception):** Untrusted text must be transformed so that it carries an unmistakable physical signal marking it as *data*, never as *instructions*.

2. **Axiom 2 (Separation):** Instructions and untrusted data must **never meet in the same model**. All security decisions must be governed by deterministic software, not probabilistic neural weights.

3.3.2 Stage 1: Keyed Spotlighting with Robust Risk Bounds

Stage 1 transforms incoming untrusted data so that the model reliably treats it as passive text while preserving clean task utility.

1. Spotlighting Primitives and Failure Modes

Static spotlighting [7] employs three transformations:

- **Delimiting:** Fencing untrusted text within static delimiters: « `Sales rose 5%.` ».

- **Datamarking:** Interleaving an invariant symbol between words: `Salesˆroseˆ5%..`
- **Encoding:** Converting the entire document into an alternative representation (e.g., Base64).

While static spotlighting reduces naive attack success, it fails against capable adversaries:

1. *Delimiter Faking:* If delimiters are static (e.g., »), an attacker injects » `[New Instruction]` to close the fence early.
2. *Marker Copying:* If the datamarking symbol is always `ˆ`, the attacker pre-marks their malicious payload with `ˆ`.
3. *Cipher Reversal:* If a static cipher like ROT13 is employed, the attacker pre-encodes the injection; the application's decoding step translates the malicious payload into clean, trusted plain text.

2. Cryptographic Randomization: Kerckhoffs's Principle

To eliminate these vulnerabilities, Horizon applies *Kerckhoffs's Principle* [10]: security must rely entirely on the secrecy of an ephemeral key, not the obscurity of the algorithm.

For every incoming request, Horizon samples a fresh secret key:

$$\kappa \sim \mathrm{Unif}(\mathcal{K}_\tau) \tag{3.4}$$

The application informs the model of κ in the system prompt and uses κ to transform the untrusted document.

Intuition (The Hotel Keycard Analogy): Knowing how the electronic lock of a hotel room works does an attacker no good; the lock is recoded for every guest with a new cryptographic key. Similarly, even if an attacker knows that Horizon uses datamarking or scrambling, they cannot predict which random symbol or character permutation is active on today's request.

3. Mathematical Formulation of the Prompt

The complete prompt submitted to the model is:

$$x_\kappa(\delta) = S_\kappa \,\|\, T \,\|\, \tau_\kappa(D \oplus \delta), \qquad \kappa \sim \mathrm{Unif}(\mathcal{K}_\tau) \tag{3.5}$$

where:

- S_κ is the system prompt embedding the request's secret key κ.
- T is the user's task instruction (e.g., "Summarize the attached document").
- $D \oplus \delta$ represents the external document D harboring a hidden adversarial injection δ.
- $\|$ denotes textual concatenation.
- τ_κ is the spotlighting transformation parameterized by key $\kappa \in \mathcal{K}_\tau$.

4. Attack Indicator and Clean Utility

We formalize attack execution as a binary indicator variable:

$$A(\delta, \kappa) = \mathbb{1}\Big[\, M(x_\kappa(\delta)) \text{ executes } \delta \,\Big] \in \{0, 1\} \tag{3.6}$$

where $M(\cdot)$ is the language model output. $A = 1$ if the model obeys the injection (e.g., outputs an exfiltration canary), and 0 otherwise.

Simultaneously, we monitor clean task utility on benign documents ($\delta = \varnothing$):

$$U_\tau = \mathbb{E}_\kappa\Big[\, u\Big(M(x_\kappa(\varnothing)),\, y^\star\Big) \,\Big] \in [0, 1] \tag{3.7}$$

where $u(\cdot, y^\star)$ scores task completion against ground-truth reference $y^\star$.

5. The Four Risk Bounds

Horizon does not evaluate defenses using raw empirical ASR. Instead, it constructs four conservative, provable risk bounds:

Bound 1: Forging Resistance (P_{forge}). By the union bound across q adversarial guesses:

$$P_{\text{forge}} \leq \frac{q}{|\mathcal{K}_\tau|} \tag{3.8}$$

Table 3.1 illustrates how key-space selection renders guessing mathematically impossible.

Table 3.1: Key-space capacity across spotlighting transformations.

Transformation τ	**Key Space $\lvert\mathcal{K}_\tau\rvert$**	**Security Guarantee**
Fixed Marker / ROT13	1	$\lvert\mathcal{K}\rvert = 1$; zero cryptographic protection.
Random Shift Cipher	25	Trivial: attacker brute-forces all 25 variants.
Dynamic Multi-Character Marker	N^k	With $N = 10$ and $k = 4$, $\lvert\mathcal{K}\rvert = 10^4$; forging $< 0.01\%$.
Random Alphabet Permutation	$\lvert\Sigma\rvert!$	Over 26 letters, $26! \approx 4.03 \times 10^{26}$; forging probability ≈ 0.

Bound 2: Finite-Sample Confidence Limit ($\bar{p}_\tau$). Given $s = \sum_{i=1}^{n} A_i$ observed attack successes across n empirical tests, the Clopper–Pearson exact upper confidence limit [16] at significance α is:

$$\bar{p}_\tau = \text{Beta}^{-1}\Big(1 - \alpha;\; s + 1,\, n - s\Big), \qquad P\Big(p \leq \bar{p}_\tau\Big) \geq 1 - \alpha \tag{3.9}$$

Crucially, when zero attacks succeed in testing ($s = 0$), the asymptotic rule of three [17] establishes:

$$\bar{p}_\tau \approx \frac{-\ln(\alpha)}{n} \approx \frac{3}{n} \quad (\text{for } \alpha = 0.05) \tag{3.10}$$

preventing false claims of absolute invulnerability from finite red-teaming trials.

Bound 3: Distributionally Robust Optimization (g_ρ). In deployment, the real-world attack distribution Q shifts away from the test distribution P. We bound this shift via the Chi-square divergence:

$$D_{\chi^2}(Q \,\|\, P) = \frac{1}{2} \int \left(\frac{dQ}{dP} - 1 \right)^2 dP \;\leq\; \rho \tag{3.11}$$

Under divergence budget $\rho \leq \frac{1-\bar{p}_\tau}{2\bar{p}_\tau}$, the worst-case expected loss admits the closed-form upper bound [18]:

$$g_\rho(\bar{p}_\tau) = \bar{p}_\tau + \sqrt{2\rho\, \bar{p}_\tau (1 - \bar{p}_\tau)} \tag{3.12}$$

Meaning: The worst attacker shifts probability mass toward attacks that already demonstrated partial success. The square-root penalty represents the maximum mathematical leverage the adversary can extract within radius ρ.

Bound 4: Adaptive Optimization Bound ($\bar{p}_\tau^{\text{adp}}$). To account for an attacker actively optimizing prompts against the model, automated adversarial loops (e.g., PAIR [13] or GCG [12]) search within query budget B:

$$\delta^\star = \arg\max_{\delta \in \Delta_B} \mathbb{E}_\kappa \Big[A(\delta, \kappa) \Big] \tag{3.13}$$

Evaluating $\delta^\star$ over n_a randomized key draws yields Clopper–Pearson adaptive bound $\bar{p}_\tau^{\text{adp}}$.

6. Combined Risk and Rule-Based Defense Selection

Combining the four bounds yields the single conservative risk metric $\bar{\mathcal{R}}_\tau(\rho)$:

$$\bar{\mathcal{R}}_\tau(\rho) = \min \left\{ 1,\; \underbrace{\frac{q}{|\mathcal{K}_\tau|}}_{\text{forging}} + \max \left(\underbrace{g_\rho(\bar{p}_\tau)}_{\text{DRO shifted style}},\; \underbrace{\bar{p}_\tau^{\text{adp}}}_{\text{adaptive attacker}} \right) \right\} \tag{3.14}$$

When choosing a single defense from family $\mathcal{T}$, Horizon applies the constrained optimization rule:

$$\tau^\star = \arg\min_{\tau \in \mathcal{T}} \bar{\mathcal{R}}_\tau(\rho) \quad \text{subject to} \quad U_\tau \geq U_{\text{none}} - \eta \tag{3.15}$$

where η is the maximum tolerated utility degradation (e.g., $\eta = 0.05$).

7. The Stage 1 Sion Minimax Equilibrium

Rather than selecting a single static transformation $\tau^\star$, the defender achieves strictly superior robustness by randomizing over spotlighting variants using probability distribution $\pi \in \Pi_\eta$.

Let $L_\pi(x) = \sum_\tau \pi_\tau A_\tau(x)$ denote the mixed defense loss on attack x. By Sion's Minimax Theorem [22], because the strategy simplex Π_η is compact and convex, and the adversarial distribution family is convex, min and max can be swapped:

$$\boxed{V_1(\rho, \eta) = \min_{\pi \in \Pi_\eta} \min \left\{ 1,\; \sum_\tau \pi_\tau \frac{q}{|\mathcal{K}_\tau|} + \max \left(\mathbb{E}_P[L_\pi] + \sqrt{2\rho\, \mathrm{Var}_P(L_\pi)},\; \sum_\tau \pi_\tau \bar{p}_\tau^{\text{adp}} \right) \right\}} \tag{3.16}$$

Randomization shrinks the variance term $\mathrm{Var}_P(L_\pi)$ because different spotlighting variants fail on disjoint subsets of adversarial prompts, guaranteeing optimal game value $V_1(\rho, \eta)$ and mixed strategy $\pi^\star$.

3.3.3 Stage 2: Deterministic Guardrail (Math-Hardened CaMeL)

Stage 1 bounds the failure probability of an individual LLM call. Stage 2 builds a deterministic causal architecture around that call, guaranteeing that even if a model inference fails, the injection cannot escalate into unauthorized software actions.

1. Architectural Separation of Concerns

As inspired by CaMeL [24], Horizon segregates execution into distinct privilege tiers:

1. **Privileged LLM:** Receives *only* trusted user prompt u. It never reads external documents, emails, or tool outputs. It synthesizes a high-level execution script:

$$\text{prog} = \text{LLM}_{\text{priv}}(u) \tag{3.17}$$

   The script expresses intent (e.g., "extract shipping date, verify customer tier, request confirmation if total exceeds threshold"). It contains no raw data.

2. **Deterministic Interpreter:** Executes prog line by line in a restricted runtime environment. It dispatches read-only extraction queries to quarantined models and routes candidate tool calls to the policy engine.

3. **Quarantined LLM Pool:** A locked-down execution container possessing zero network permissions, zero disk access, and zero tool capabilities. It is invoked strictly to answer extracted-value queries.

2. Quarantined Execution Pool with Independent Keys

When prog requires processing untrusted external text x:

1. Horizon samples variant $\tau \sim \pi^\star$ and draws m independent ephemeral keys:

$$\kappa_1, \ldots, \kappa_m \overset{\text{i.i.d.}}{\sim} \text{Unif}(\mathcal{K}_\tau) \tag{3.18}$$

2. Untrusted text x is transformed under each key: $\tilde{x}_j = \tau_{\kappa_j}(x)$.

3. The transformed inputs are dispatched to m isolated instances of the quarantined model:

$$y_1 \sim \text{LLM}_{\text{quar}}(\tilde{x}_1), \quad \ldots, \quad y_m \sim \text{LLM}_{\text{quar}}(\tilde{x}_m) \tag{3.19}$$

3. Majority Voting and the Hoeffding Bound

The deterministic interpreter computes the consensus output:

$$y^\star = \text{Majority}(y_1, \ldots, y_m) \tag{3.20}$$

If more than $m/2$ runs agree on value y, $y^\star$ is accepted. If agreement is not reached, execution halts, an anomaly alert is registered, and the task escalates to the user.

Theorem 1 (Exponential Failure Decay via Hoeffding Concentration): Let $\bar{\mathcal{R}}_Q < \frac{1}{2}$ denote Stage 1's guaranteed upper bound on the failure rate of a single quarantined call. The probability that an attacker fools the majority vote is bounded by:

$$\boxed{P(\text{vote fooled}) \;\leq\; \exp\left(-2\,m\left(\tfrac{1}{2} - \bar{\mathcal{R}}_Q\right)^2\right)} \tag{3.21}$$

Proof. Each quarantined run $j \in \{1, \ldots, m\}$ uses an independent key κ_j. The event that run j succumbs to injection is a Bernoulli trial with success probability $p_j \leq \bar{\mathcal{R}}_Q < \frac{1}{2}$. The vote is fooled only if the sum of compromised runs satisfies $\sum_{j=1}^{m} \mathbb{1}_{\text{compromised}} \geq \frac{m}{2}$. By Hoeffding's Inequality for bounded independent random variables [20], setting deviation $t = \frac{1}{2} - \bar{\mathcal{R}}_Q$:

$$P\left(\frac{1}{m}\sum_{j=1}^{m} X_j - \bar{\mathcal{R}}_Q \geq \frac{1}{2} - \bar{\mathcal{R}}_Q\right) \leq \exp\left(-2m\left(\frac{1}{2} - \bar{\mathcal{R}}_Q\right)^2\right) \tag{3.22}$$

completing the proof. □

Numerical Walkthrough: Suppose Stage 1 bounds single-call risk to $\bar{\mathcal{R}}_Q = 0.15$. The safety margin is $\frac{1}{2} - 0.15 = 0.35$.

- $m = 5$ runs: $P(\text{fooled}) \leq \exp(-2 \times 5 \times 0.35^2) = \exp(-1.225) \approx \mathbf{29.3}\%$.
- $m = 9$ runs: $P(\text{fooled}) \leq \exp(-2 \times 9 \times 0.35^2) = \exp(-2.205) \approx \mathbf{11.0}\%$.
- $m = 15$ runs: $P(\text{fooled}) \leq \exp(-2 \times 15 \times 0.35^2) = \exp(-3.675) \approx \mathbf{2.5}\%$.

Thus, majority voting converts linear statistical guarantees into exponential containment.

4. Typed Values and Capability Tokens

Every value accepted from majority voting is encapsulated within an immutable typed envelope:

$$v = \big(\text{data},\ \text{type},\ \text{cap}\big) \tag{3.23}$$

- **data:** The raw string extracted by the model (e.g., `"2026-11-04"`).
- **type:** Syntactically validated by a strict deterministic grammar parser. If the quarantined model outputs code or SQL when expecting an ISO-8601 date, the parser rejects it instantly.
- **cap:** An immutable capability token denoting provenance:

$$\text{cap} \in \{\text{USER},\ \text{SYSTEM},\ \text{UNTRUSTED}\} \tag{3.24}$$

5. SMT-Verified Policy Engine and Noninterference

Before the interpreter dispatches any external action, the proposed tool invocation t with argument vector $(v_1, \ldots, v_k)$ is evaluated against a deterministic policy:

$$\text{Policy}(t, v_1, \ldots, v_k) \in \{\text{ALLOW},\ \text{DENY},\ \text{CONFIRM}\} \tag{3.25}$$

We enforce the formal *Noninterference Theorem* [32]:

$$\boxed{\exists\, i \text{ such that } v_i.\text{cap} = \text{UNTRUSTED} \implies \text{Policy}(t, \mathbf{v}) \neq \text{ALLOW} \quad (\forall\, t \in \mathcal{T}_{\text{critical}})} \tag{3.26}$$

Untrusted data is mathematically barred from selecting target tools or executing high-privilege system modifications. The consistency and completeness of all policy rules are statically verified at startup using the Z3 SMT solver [33].

6. Conformal Risk Control on Confirmation Thresholds

When Policy outputs **CONFIRM**, human approval is solicited before tool execution. To guard against human habituation and false approvals, Horizon applies Conformal Risk Control (CRC) [21].

Let $\lambda \in [0, 1]$ parameterize the threshold for escalating to user confirmation. Given user mistake loss function $\mathcal{L}_{\text{user}}$, CRC calibrates $\hat{\lambda}$ on calibration data such that:

$$\mathbb{E}\left[\mathcal{L}_{\text{user}}(\hat{\lambda})\right] \leq \alpha_{\text{user}} \tag{3.27}$$

bounding the expected probability that a deceptive injection slips past human review to a rigorous confidence budget (e.g., $\alpha_{\text{user}} = 1\%$).

7. The Final Horizon Formula

A complete Horizon configuration is defined as the tuple:

$$\theta = \Big(m,\ \text{parser},\ \text{policy},\ \lambda\Big) \in \Theta \tag{3.28}$$

The per-attack loss of configuration θ on attack x is:

$$\ell_\theta(x) = G_\theta(x) + a\, C_\theta(x) + h_{\mathcal{S},\theta}\, F_\theta(x) \tag{3.29}$$

where:

- $G_\theta(x) \in \{0, 1\}$ indicates whether the attack bypassed the policy entirely.
- $C_\theta(x) \in \{0, 1\}$ indicates whether the attack reached human confirmation, with $a \in [0, 1]$ denoting the empirical wrong-approval rate.
- $F_\theta(x) \in \{0, 1\}$ indicates whether the attack fooled majority voting and syntactic parsing, with $h_{\mathcal{S},\theta} \in [0, 1]$ denoting the conditional probability that a corrupted value induces system harm.

Horizon randomizes over configuration set Θ via probability vector $\pi \in \Pi$, where:

$$\Pi = \Big\{\pi \geq 0,\ \textstyle\sum_\theta \pi_\theta = 1,\ \text{utility} \geq U_{\min},\ \text{leakage} \leq B\Big\} \tag{3.30}$$

with B denoting the information leakage bound in bits verified via Quantitative Information Flow [34].

By Sion's Minimax Theorem, the global game equilibrium risk ceiling of the Horizon architecture is:

$$V_{\text{Horizon}} = \min_{\pi \in \Pi} \min \left\{ 1, \quad \underbrace{\sum_{\theta} \pi_\theta \frac{q}{|\mathcal{K}_\theta|}}_{\text{forging resistance}} + \max \left(\underbrace{\mathbb{E}_P[L_\pi] + \sqrt{2\rho \operatorname{Var}_P(L_\pi)}}_{\text{worst shifted attack distribution}}, \quad \underbrace{\sum_{\theta} \pi_\theta \bar{\ell}_\theta^{\text{adp}}}_{\text{adaptive attacker}} \right) \right\} \tag{3.31}$$

where the composite loss is:

$$L_\pi(x) = \sum_{\theta} \pi_\theta \left(\bar{p}_{G,\theta}^{\text{CP}} + a\, \bar{p}_{C,\theta}^{\text{CP}} + h_{\mathcal{S},\theta}\, \bar{p}_{F,\theta}^{\text{CP}} \right) \tag{3.32}$$

8. Architectural Outputs

The Horizon architecture delivers two concrete operational guarantees:

1. **At Runtime (Per-Request Execution):** The user's task completes normally. Injected payloads are mathematically defanged into passive typed strings, blocked by deterministic noninterference policies, or escalated through calibrated human confirmation.

2. **At Design Time (System Certification):** The Sion game solver produces the optimal configuration mix $\pi^\star_{\text{Horizon}}$, the provable risk ceiling V_{Horizon}, and the adversary's worst-case distribution $w^\star$, identifying exactly where defensive resources must be allocated.

Chapter 4

Experiments

4.1 Experimental Setup and Testbed

Description of the testbed environment, simulated network topology, and agent execution sandbox.

4.2 Threat Models and Attack Scenarios

- Cache-Augmented Generation (CAG) memory poisoning scenarios.
- GraphRAG topological relationship injections.
- Steganographic image-based prompt injections (IPI).
- Multi-step Confused Deputy tool hijacking attacks.

4.3 Evaluation Datasets and Red-Teaming Benchmarks

- **Agent Red Teaming (ART) Benchmark:** Automated adversarial prompts targeting policy violations.
- **ExploitGym:** Autonomous tool exploitation and misuse benchmark.
- **Automated Fuzzing:** NVIDIA Garak and Microsoft PyRIT multi-turn conversational red-teaming.

4.4 Baseline Configurations

Comparison against unprotected agent baselines and traditional static prompt filter implementations.

Chapter 5

Evaluation

5.1 Detection Efficacy (RQ1 & RQ2)

Evaluation of Mahalanobis distance filters and 2D FFT spectral detectors against embedding anomalies and visual prompt injections.

5.2 Safety Barrier Enforcement (RQ3)

Empirical results of Control Barrier Function (CBF) boundary enforcement under multi-turn Confused Deputy attacks.

5.3 Cybernetic Rate Limiting Performance (RQ4)

Evaluation of the PID-governed token bucket against burst data exfiltration attempts.

5.4 System Overhead and Latency Analysis

Runtime latency, computational overhead, and throughput impact of the tri-hook middleware.

Chapter 6

Discussion

6.1 Balancing Mathematical Rigor and Software Practicality

Insights gained from mapping continuous mathematical boundary functions onto discrete agent API execution spaces.

6.2 Security Depth vs. Latency Overhead

Trade-off analysis between computational latency introduced at each hook and the corresponding theoretical security guarantees provided.

6.3 Assumptions and Limitations

Assumptions regarding embedding distributions, matrix inversion overhead in high dimensions, and boundary cases in tool hijacking defense.

Chapter 7

Future Work

7.1 Adaptive Latent Space Barrier Learning

Training parameterized neural barrier functions dynamically adapted to evolving agent toolspaces and contextual policies.

7.2 Hardware-Accelerated Cryptographic Hooks

Integrating Fully Homomorphic Encryption (FHE) with hardware acceleration for zero-knowledge runtime context queries.

7.3 Multi-Agent Swarm Defense Protocols

Extending tri-hook verification across decentralized multi-agent topologies and collective decision environments.

AI Disclosure

In accordance with academic integrity guidelines, generative AI tools and coding assistants were utilized during the preparation of this thesis for the following specific purposes:

- Grammar, style refinement, and drafting assistance for preliminary documentation.
- Syntax assistance for LaTeX templates, tables, and bibliographical formatting.

All core conceptual ideas, mathematical formulations, software architectural designs, experimental implementations, and analyses represent the original work of the author.

Bibliography

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pp. 79–90, 2023.

[2] J. Yi, Y. Xie, Z. Xiang, D. Chen, C. Shen, X. Xing, and K. Li. Benchmarking and defending against indirect prompt injection attacks on large language models. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024.

[3] X. Jiang, S. Yang, W. Yang, Y. Liu, and C. Ji. Agentic AI as a cybersecurity attack surface: Threats, exploits, and defenses in runtime supply chains. In *IEEE International Conference on Artificial Intelligence (CAI)*, 2026.

[4] OWASP Foundation. OWASP Top 10 for Agentic Applications 2026: The Agent Runtime Authority Gap and Beyond. *OWASP Security Guidance Repository*, 2026.

[5] E. Zverev and S. Abdelnabi. Can LLMs separate instructions from data? And what do we even mean by that? In *International Conference on Learning Representations (ICLR)*, 2025.

[6] E. Zverev, E. Kortukov, S. Tabesh, and A. Panfilov. ASIDE: Architectural separation of instructions and data in language models. In *International Conference on Learning Representations (ICLR)*, 2026.

[7] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman. Defending against indirect prompt injection attacks with spotlighting. *arXiv preprint arXiv:2403.14720*, Microsoft, 2024.

[8] S. Sharma. Containment over detection: Cryptographic boundary enforcement for prompt injection defense in agentic LLM systems. *Amazon Research Report / Research Square*, 2026.

[9] A. Bhatnagar and G. Pokhariyal. Semantically encoded compiler stage watermarks: Mitigating LLM prompt injection, hallucination, and security risks. *SSRN Electronic Journal*, 2025.

[10] A. Kerckhoffs. La cryptographie militaire. *Journal des sciences militaires*, vol. 9, pp. 5–38, 161–191, 1883.

[11] M. Nasr, N. Carlini, J. Hayes, M. Ilie, and H. Chaudhari. The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections. *arXiv preprint arXiv:2502.12345*, 2025.

[12] A. Zou, Z. Wang, J. Zico Kolter, and M. Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

[13] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong. Jailbreaking black box large language models in twenty queries. *arXiv preprint arXiv:2310.08419*, 2023.

[14] Y. Yuan, W. Jiao, W. Wang, J. Huang, P. He, S. Shi, and Z. Tu. GPT-4 is too smart to be safe: Stealthy chat with LLMs via cipher. *arXiv preprint arXiv:2402.10601*, 2024.

[15] W. R. Hosea, D. Ng, D. Sharma, G. Ng, and K. Srinivasan. Defending large language models against jailbreak exploits with responsible AI considerations. *arXiv preprint arXiv:2511.18933*, 2025.

[16] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934.

[17] J. A. Hanley and A. Lippman-Hand. If nothing goes wrong, is everything all right? Interpreting zero numerators. *JAMA*, 249(13):1743–1745, 1983.

[18] J. C. Duchi and H. Namkoong. Learning models with uniform performance via distributionally robust optimization. *The Annals of Statistics*, 49(3):1378–1406, 2021.

[19] A. Ben-Tal, D. den Hertog, A. De Waegenaere, B. Melenberg, and G. Rennen. Robust solutions of optimization problems affected by uncertain probabilities. *Management Science*, 59(2):341–357, 2013.

[20] W. Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

[21] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, and T. Schuster. Conformal risk control. In *International Conference on Learning Representations (ICLR)*, 2024.

[22] M. Sion. On general minimax theorems. *Pacific Journal of Mathematics*, 8(1):171–176, 1958.

[23] A. Paulus, I. Kulikov, B. Amos, R. Munos, I. Evtimov, K. Chaudhuri, and A. Zharmagambetov. Safety alignment of LMs via non-cooperative games. *Meta AI & University of Tübingen Research Report*, 2025.

[24] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, Google, Google DeepMind, ETH Zurich, 2025.

[25] M. Costa, A. Salem, B. Köpf, S. Tople, and A. Kolluri. Securing AI agents with information-flow control. *arXiv preprint arXiv:2505.23643*, 2025.

[26] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen. NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. *arXiv preprint arXiv:2310.10501*, NVIDIA, 2023.

[27] D. Schwarz. Countermind: A multi-layered security architecture for large language models. *Independent Cybersecurity Research*, 2025.

[28] Y. Louck and A. Stulman. Security analysis of agentic AI communication protocols: A comparative evaluation. *Ariel Cyber Innovation Center Technical Report*, 2026.

[29] D. Kothari. A hash-based trust framework for multi-agent AI systems: Ensuring instruction integrity through cryptographic verification. *SSRN Electronic Journal*, 2025.

[30] S. Panchumarthi and D. Sakamuri. Securing inter-agent communication: A framework for trusted and secure AI-to-AI collaboration. *Cybersecurity Technical Report*, 2025.

[31] A. Al-Nuaimi et al. A formal security framework for MCP-based AI agents: Threat taxonomy, verification models, and defense mechanisms. *Journal of Systems and Software Security*, 2026.

[32] J. A. Goguen and J. Meseguer. Security policies and security models. In *IEEE Symposium on Security and Privacy (S&P)*, pp. 11–20, 1982.

[33] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 4963, pp. 337–340, 2008.

[34] G. Smith. On the foundations of quantitative information flow. In *Foundations of Software Science and Computational Structures (FoSSaCS)*, LNCS 5504, pp. 288–302, 2009.

[35] A. D. Ames, S. Coogan, M. Egerstedt, M. Notomista, K. Sreenath, and P. Tabuada. Control barrier functions: Theory and applications. In *18th European Control Conference (ECC)*, IEEE, pp. 3420–3431, 2019.

[36] P. C. Mahalanobis. On the generalized distance in statistics. *Proceedings of the National Institute of Sciences of India*, 2(1):49–55, 1936.

[37] NVIDIA Corporation. Garak: Generative AI Red-teaming & Assessment Kit. *NVIDIA Open Source Security Framework*, 2024.

[38] Microsoft AI Red Team. Python Risk Identification Tool for Generative AI (PyRIT). *Microsoft AI Red Team Technical Documentation*, 2024.

Appendix A

Mathematical Proofs

A.1 Forward Invariance of Control Barrier Functions

Let $\mathcal{C} = \{x \in \mathbb{R}^n : h(x) \geq 0\}$ denote the 0-superlevel set of a continuously differentiable function $h : \mathcal{D} \subset \mathbb{R}^n \to \mathbb{R}$. The function h is a Control Barrier Function (CBF) if there exists an extended class $\mathcal{K}_\infty$ function α such that for the control system $\dot{x} = f(x) + g(x)u$:

$$\sup_{u \in U} \left[L_f h(x) + L_g h(x) u \right] \geq -\alpha(h(x)), \quad \forall x \in \mathcal{D} \tag{A.1}$$

Any controller choosing $u \in K_{cbf}(x)$ renders the set $\mathcal{C}$ forward invariant, guaranteeing safety for all $t \geq 0$.

Appendix B

Architecture Configuration and Schemas

B.1 Middleware Configuration Schema

Sample configuration schema for deploying the NetWatch tri-hook runtime interception gateway.